UNIVERSITAS NEGERI YOGYAKARTA
FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK INDUSTRI

LAPORAN PRAKTIKUM

APLIKASI WEB DAN MOBILE

Semester Genap 2025/2026

MODUL 10 – State, Hooks (useState, useEffect), dan Conditional Rendering

Nama Mahasiswa	:	Nailah Azka Iftinah
NIM	:	23051430026
Kelas	:	A
Tanggal Praktikum	:	26 April 2026
Dosen Pengampu	:	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT.

Dikumpulkan paling lambat 7 hari setelah pelaksanaan praktikum

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 10 (Pertemuan 10)
Topik Utama	State, Hooks (useState, useEffect), dan Conditional Rendering
Sub-Topik	State Management, Lifecycle Hooks, dan Rendering Bersyarat.
Durasi Praktikum	340 Menit
Tanggal Praktikum	26 April 2026
Nama Mahasiswa	Nailah Azka Iftinah
NIM	23051430026
Kelas	A

B. TUJUAN PEMBELAJARAN

Tuliskan tujuan pembelajaran yang ingin dicapai pada modul ini, sesuai dengan yang tertera di Manual Praktikum.

1	Memahami apa itu State dan perbedaannya dengan Props.
2	Menguasai penggunaan `useState` untuk menyimpan data yang berubah-ubah.
3	Memahami `useEffect` untuk menangani side effects (seperti timer atau API call).
4	Mampu melakukan Conditional Rendering (menyembunyikan/menampilkan elemen berdasarkan kondisi).

C. ALAT DAN BAHAN

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern (Chrome/Firefox/Edge).	Versi terbaru	Testing & Debugging
3	Ekstensi VS Code	*Live Server & CodeSnap*	Menjalankan HTML di server lokal dan memperbarui

			tampilan otomatis saat file berubah. Mengubah kode program menjadi gambar untuk dokumentasi
5	Git/Github	Versi Terbaru	Repository dan Manajemen versi
6	JavaScript (Client-Side)	ES6 (Modern JavaScript)	Bahasa pemrograman untuk logika & perhitungan
7	Quokka.js	Extension VS Code	Untuk melihat hasil JavaScript secara realtime
8	Node.js	LTS	Menjalankan react

D. DASAR TEORI

State dalam React adalah data yang dapat berubah selama aplikasi berjalan dan dikelola oleh komponen itu sendiri. Berbeda dengan props yang hanya bersifat read-only (hanya bisa dibaca), state bersifat read-write (dapat diubah). State berfungsi sebagai “memori sementara” dalam komponen untuk menyimpan nilai yang dinamis, seperti jumlah produksi, status mesin, atau input pengguna. Ketika nilai state berubah, React secara otomatis akan melakukan proses re-render, yaitu memperbarui tampilan antarmuka agar sesuai dengan data terbaru. Konsep ini sangat penting dalam pengembangan aplikasi interaktif karena memungkinkan sistem merespon perubahan secara langsung tanpa perlu memuat ulang halaman.

useState merupakan salah satu hook dalam React yang digunakan untuk membuat dan mengelola state dalam functional component. Hook ini memungkinkan developer untuk menyimpan nilai dalam suatu variabel state dan menyediakan fungsi untuk memperbaruinya. Sintaks dasar useState adalah const [state, setState] = useState(nilaiAwal). Ketika fungsi setter dipanggil, React akan memperbarui nilai state dan otomatis merender ulang komponen. Dalam konteks aplikasi industri, useState dapat digunakan untuk menyimpan data seperti jumlah produksi, target, atau status proses yang terus berubah.

useEffect adalah hook yang digunakan untuk menangani side effects dalam komponen React, seperti menjalankan timer, mengambil data dari API, atau melakukan manipulasi terhadap DOM. useEffect dapat dijalankan pada berbagai kondisi, seperti saat komponen pertama kali dimuat (mount), saat terjadi perubahan state atau props (update), maupun saat komponen akan dihapus (unmount). Penggunaan dependency array pada useEffect menentukan kapan fungsi tersebut dijalankan. Jika dependency array kosong ([]), maka useEffect hanya akan berjalan sekali saat komponen pertama kali ditampilkan. Selain itu, useEffect juga mendukung cleanup function yang berguna untuk menghentikan proses seperti timer agar tidak terus berjalan dan menyebabkan memory leak.

Conditional Rendering adalah teknik dalam React yang digunakan untuk menampilkan atau menyembunyikan elemen berdasarkan kondisi tertentu. Teknik ini dapat dilakukan menggunakan berbagai cara seperti pernyataan if, operator ternary (? :), atau logical operator (&&). Dengan conditional rendering, tampilan aplikasi dapat berubah secara dinamis sesuai dengan kondisi data, misalnya menampilkan pesan “Target Tercapai” ketika jumlah produksi sudah memenuhi target atau “Produksi Berjalan” ketika target belum tercapai. Hal ini membuat aplikasi menjadi lebih responsif dan informatif.

Penggabungan konsep state, useState, useEffect, dan conditional rendering memungkinkan pengembangan aplikasi yang interaktif dan dinamis. Dalam satu alur kerja, state menyimpan data yang berubah, useState digunakan untuk mengelola perubahan tersebut, useEffect menjalankan proses tambahan seperti timer atau pembaruan data, dan conditional rendering menentukan tampilan yang sesuai dengan kondisi. Kombinasi ini menjadi dasar dalam membangun aplikasi berbasis React yang mampu menampilkan data secara real-time, efisien, dan sesuai dengan kebutuhan sistem modern, khususnya dalam pengembangan dashboard atau sistem monitoring di industri...

D.1 Konsep Utama

Pada praktikum pertemuan kesepuluh ini, pembelajaran memasuki tahap pengembangan aplikasi React yang lebih interaktif dengan memanfaatkan state dan hooks. Jika pada pertemuan sebelumnya aplikasi hanya berfokus pada tampilan menggunakan props, pada pertemuan ini diperkenalkan mekanisme pengelolaan data yang dapat berubah secara langsung di dalam komponen. Hal ini memungkinkan aplikasi untuk merespon interaksi pengguna secara real-time tanpa perlu memuat ulang halaman, sehingga sesuai dengan kebutuhan sistem modern seperti dashboard monitoring produksi berbasis Single Page Application (SPA).

Konsep pertama yang dipelajari adalah State. Dalam React, state merupakan data yang dapat berubah selama aplikasi berjalan dan dimiliki oleh komponen itu sendiri. State

digunakan untuk menyimpan nilai seperti jumlah produksi, target, atau status sistem. Ketika state berubah, React akan secara otomatis memperbarui tampilan agar sesuai dengan kondisi terbaru. Hal ini membuat aplikasi menjadi lebih dinamis dan responsif terhadap perubahan data.

Konsep kedua adalah useState. useState merupakan hook yang digunakan untuk membuat dan mengelola state dalam functional component. Dengan useState, developer dapat mendefinisikan nilai awal suatu data dan memperbaruinya melalui fungsi setter. Pada praktikum ini, useState digunakan untuk menambah jumlah produksi, mengatur target, serta melakukan reset. Hal ini menunjukkan bahwa data dalam aplikasi dapat berubah sesuai dengan interaksi pengguna.

Konsep ketiga adalah useEffect. useEffect digunakan untuk menjalankan proses tambahan atau side effects dalam komponen, seperti menjalankan timer atau melakukan update data secara otomatis. Pada praktikum ini, useEffect digunakan untuk membuat jam digital yang terus berjalan setiap detik. Dengan adanya dependency array, developer dapat mengatur kapan fungsi tersebut dijalankan, sehingga aplikasi tetap efisien dan terhindar dari error seperti infinite loop.

Konsep keempat adalah Conditional Rendering. Conditional rendering merupakan teknik untuk menampilkan atau menyembunyikan elemen berdasarkan kondisi tertentu. Dalam praktikum ini, digunakan untuk menampilkan pesan yang berbeda tergantung pada apakah jumlah produksi sudah mencapai target atau belum. Dengan teknik ini, tampilan aplikasi dapat menyesuaikan kondisi data secara langsung dan memberikan informasi yang lebih jelas kepada pengguna.

Konsep kelima adalah Interaktivitas Aplikasi. Dengan adanya state dan hooks, aplikasi menjadi lebih interaktif karena pengguna dapat langsung berinteraksi melalui tombol atau input. Setiap aksi pengguna akan mempengaruhi state dan otomatis mengubah tampilan. Hal ini sangat penting dalam pengembangan sistem modern yang membutuhkan respon cepat terhadap perubahan data.

Terakhir adalah Alur Kerja State dan Hooks dalam React. Dalam pengembangan aplikasi, alur dimulai dari deklarasi state menggunakan useState, kemudian perubahan state melalui interaksi pengguna, dilanjutkan dengan penggunaan useEffect untuk menjalankan proses tambahan, dan diakhiri dengan tampilan yang diperbarui melalui conditional rendering. Pendekatan ini memungkinkan aplikasi berjalan secara dinamis, efisien, dan mendukung kebutuhan sistem real-time seperti dashboard monitoring dalam industri.

D.2 Keterkaitan dengan Teknik Industri

Jika dikaitkan dengan Teknik Industri, materi pada pertemuan kesepuluh ini sangat penting karena memperkenalkan konsep pengelolaan data dinamis dan interaksi sistem secara real-time yang menjadi bagian utama dalam implementasi sistem digital di era Industri 4.0. Dalam lingkungan manufaktur modern, data seperti jumlah produksi, status mesin, dan waktu operasional selalu berubah dan harus ditampilkan secara cepat dan akurat. Penggunaan state dan hooks dalam React memungkinkan sistem untuk memperbarui data secara otomatis tanpa perlu memuat ulang halaman, sehingga mendukung pengambilan keputusan yang lebih cepat dan efisien.

Konsep State dalam React dapat dianalogikan sebagai kondisi nyata dalam sistem produksi yang selalu berubah. Misalnya, jumlah output produksi akan terus bertambah seiring waktu, dan sistem harus mampu menampilkan perubahan tersebut secara langsung. Dengan state, data dapat disimpan dan diperbarui secara fleksibel, sehingga mencerminkan kondisi aktual di lapangan tanpa harus menghentikan sistem. Penggunaan useState mencerminkan bagaimana sistem industri mengelola perubahan data berdasarkan input atau kejadian tertentu. Dalam dunia industri, perubahan data bisa berasal dari operator maupun sensor mesin. Dengan useState, setiap perubahan seperti penambahan produksi atau reset sistem dapat langsung diproses dan ditampilkan. Hal ini sejalan dengan prinsip responsivitas dalam sistem produksi modern.

Konsep useEffect memiliki keterkaitan dengan sistem monitoring otomatis dalam industri, seperti sensor atau sistem SCADA yang terus memperbarui data secara berkala. Pada praktikum ini, useEffect digunakan untuk membuat jam digital yang berjalan setiap detik, yang mencerminkan bagaimana sistem dapat mengambil dan memperbarui data secara terus-menerus di latar belakang tanpa mengganggu proses utama. Konsep Conditional Rendering mencerminkan sistem pengambilan keputusan berbasis kondisi dalam industri. Misalnya, ketika target produksi tercapai, sistem akan menampilkan notifikasi atau indikator tertentu. Sebaliknya, jika target belum tercapai, sistem akan menunjukkan bahwa proses masih berjalan. Hal ini membantu operator atau manajer dalam memahami kondisi sistem secara cepat dan mengambil tindakan yang tepat.

Terakhir, penerapan state dan hooks dalam React sangat relevan dengan pengembangan sistem dashboard berbasis Single Page Application (SPA) yang banyak digunakan dalam industri modern. Dashboard produksi, sistem monitoring, dan aplikasi operasional harus mampu menampilkan data secara real-time, interaktif, dan responsif. Dengan memahami konsep ini, mahasiswa Teknik Industri dapat merancang sistem yang

tidak hanya mampu menampilkan data, tetapi juga merespon perubahan secara langsung dan mendukung proses pengambilan keputusan yang efektif di lingkungan industri.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

Dokumentasikan setiap langkah kerja yang Anda lakukan, beserta hasil yang diperoleh (screenshot, kode, atau output). Jawab sesuai urutan langkah di Manual Praktikum.

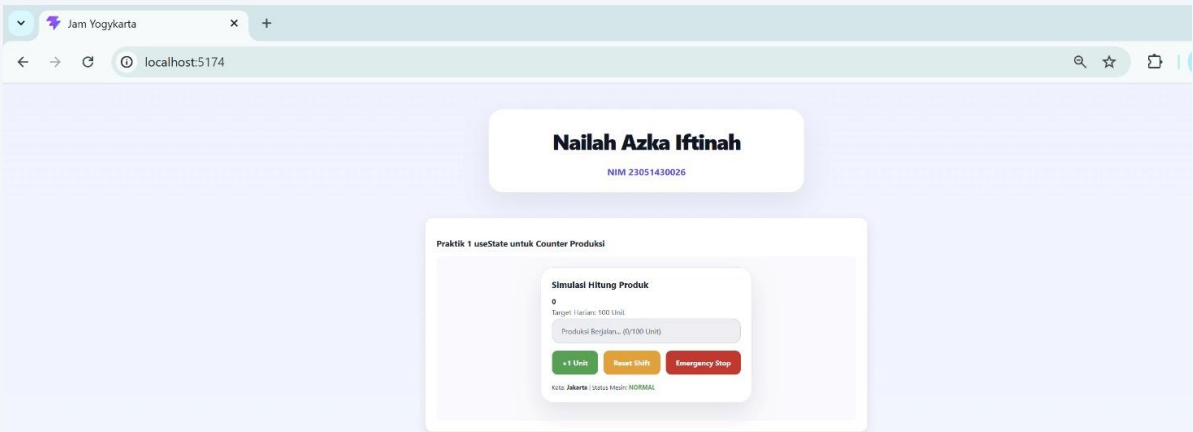
E.1 Langkah 1 – Membuat Komponen Counter Produksi

Buat file baru `src/Komponen/CounterProduksi.jsx`.

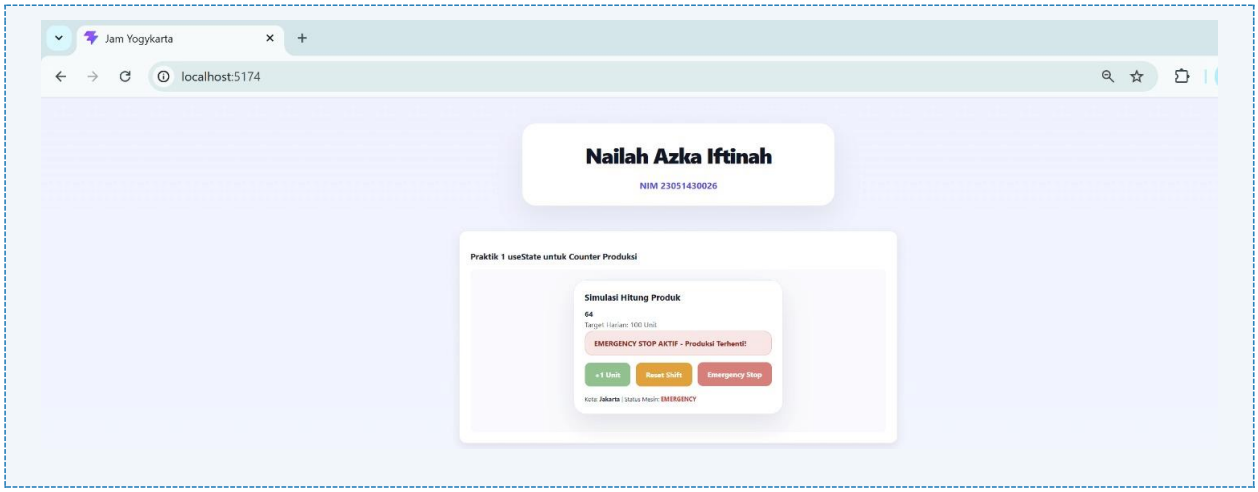
```
/* ======= KODE / OUTPUT LANGKAH 1=======
import React, { useState } from 'react';

function CounterProduksi() {
  // Deklarasi State: 'jumlah' bernilai awal 0, 'setJumlah' fungsi untuk
mengubahnya
  const [jumlah, setJumlah] = useState(0);
  const [target, setTarget] = useState(100);

  // Fungsi menambah produksi
  const tambahProduksi = () => {
    setJumlah(jumlah + 1);
  };

  // Fungsi reset
  const reset = () => {
    setJumlah(0);
  };

  return (
    <div className="text-center p-4 border rounded bg-light">
      <h3>Simulasi Hitung Produk</h3>
      <h1 className="display-4">{jumlah}</h1>
      <p>Target: {target} Unit</p>

      {/* Conditional Rendering: Tampilkan pesan jika target tercapai */}
      {jumlah >= target ? (
        <div className="alert alert-success d-inline-block">
          Target Tercapai!
        </div>
      ) : (
        <div className="alert alert-secondary d-inline-block">
          Produksi Berjalan...
        </div>
```

```
      )}

      <div className="mt-3">
        <button
          className="btn btn-primary me-2"
          onClick={tambahProduksi}
        >
          +1 Unit (Sensor OK)
        </button>

        <button
          className="btn btn-danger"
          onClick={reset}
        >
          Reset Shift
        </button>
      </div>
    </div>
  );
}

export default CounterProduksi;
```

Screenshot Hasil:



Gambar E.1– Hasil Membuat Komponen Counter Produksi



Gambar E.1– Hasil Membuat Komponen Counter Produksi

E.2 Langkah 2 – Menggunakan useEffect untuk Real-time Clock

Kita akan membuat jam digital yang berjalan otomatis. Ini mensimulasikan pengambilan data live dari sensor tiap detik. Tambahkan kode di dalam `CounterProduksi.jsx` (atau buat komponen baru `JamDigital.jsx`):

```
/* ======= KODE / OUTPUT LANGKAH 2=======
import React, { useState, useEffect } from 'react';

function JamDigital() {
  const [waktu, setWaktu] = useState(new Date());

  // useEffect berjalan sekali setelah komponen dirender pertama kali
  useEffect(() => {
    // Membuat interval timer
    const timerID = setInterval(() => {
      setWaktu(new Date()); // Update state waktu setiap detik
    }, 1000);

    // Cleanup function: Dijalankan saat komponen dihapus/hancur
    // Penting untuk mencegah memory leak
    return () => {
      clearInterval(timerID);
    };
  }, []); // Array kosong [] artinya hanya dijalankan sekali saat mount

  return (
    <div className="alert alert-info text-center">
      <h4>Waktu Sistem Server: {waktu.toLocaleTimeString()}</h4>
    </div>
  );
}
```

```
export default JamDigital;
```

Screenshot Hasil:

Gambar E.2 – Hasil Menggunakan useEffect untuk Real-time Clock

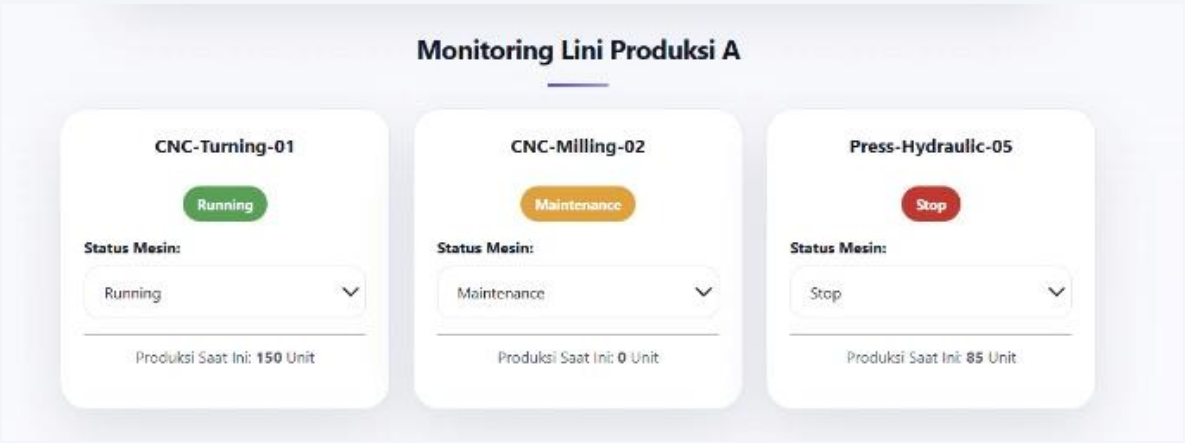
E.3 Langkah 3 – Contoh Form Input State

Ubah komponen `KartuMesin.jsx` dari pertemuan 9 agar statusnya bisa diedit secara lokal (opsional latihan):

```
/* ======= KODE / OUTPUT LANGKAH 3 ======= */
// Di dalam KartuMesin.jsx
const [statusLokal, setStatusLokal] = useState(status);

return (
  // ... kode HTML sebelumnya ...

  <div className="mt-2">
    <select
      className="form-select form-select-sm"
      value={statusLokal}
      onChange={(e) => setStatusLokal(e.target.value)}
    >
      <option value="Running">Running</option>
      <option value="Stop">Stop</option>
      <option value="Maintenance">Maintenance</option>
    </select>
```

```
  </div>

  // ...
)
```

Screenshot Hasil:



Gambar E.3– Hasil Contoh Form Input State

F. LATIHAN DAN TUGAS MANDIRI

Jawab dan dokumentasikan seluruh latihan/tugas yang ada di bagian "Latihan/Tugas Mandiri" pada modul Anda.

F.1 Latihan 1 – (Dependency Array useEffect):

Pertanyaan / Instruksi Latihan:

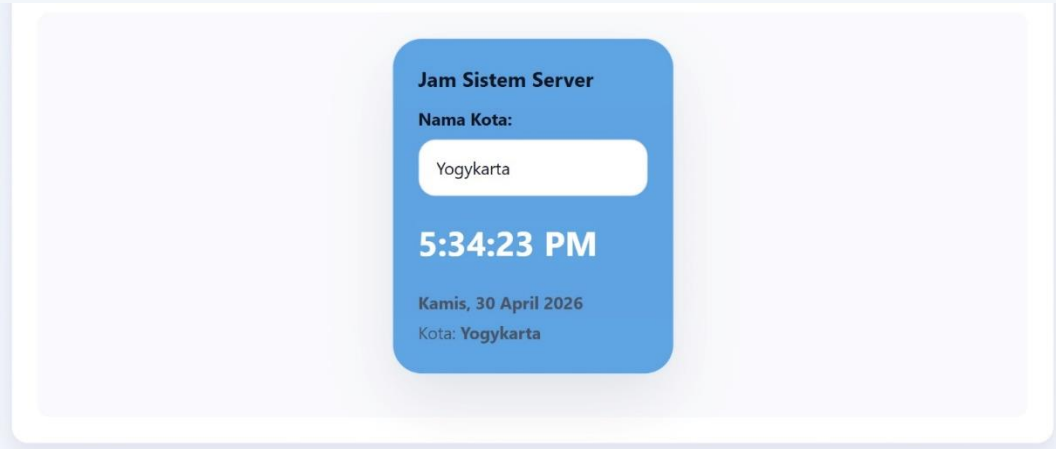
> Latihan 1 (Dependency Array useEffect): Modifikasi `JamDigital`. Tambahkan input text untuk memasukkan nama kota. Gunakan `useEffect` untuk mengubah `document.title` menjadi "Jam [Nama Kota]". Gunakan state kota sebagai dependency array `[kota]`.

Jawaban / Kode Solusi:

```
import React, { useState, useEffect } from 'react';

function JamDigital() {
  // State waktu
  const [waktu, setWaktu] = useState(new Date());

  // State kota
  const [kota, setKota] = useState('Yogyakarta');

  // useEffect untuk jam real-time
  useEffect(() => {
    const timerID = setInterval(() => {
      setWaktu(new Date());
    }, 1000);

    return () => {
      clearInterval(timerID);
    };
  }, []);

  // useEffect untuk ubah title sesuai kota
  useEffect(() => {
    document.title = `Jam ${kota}`;
  }, [kota]);

  return (
    <div className="text-center p-4 border rounded bg-light">
      <h3>Jam Digital</h3>

      <h1 className="display-5">
        {waktu.toLocaleTimeString()}
      </h1>
```

```
      <p>Kota: <strong>{kota}</strong></p>

      <input
        type="text"
        className="form-control mt-2"
        value={kota}
        onChange={(e) => setKota(e.target.value)}
        placeholder="Masukkan nama kota"
      />
    </div>
  );
}

export default JamDigital;
```

Screenshot Hasil:



Gambar F.1 – Hasil Latihan 1

F.2 Latihan 2 – (Conditional Rendering):

Pertanyaan / Instruksi Latihan:

> Latihan 2 (Conditional Rendering): Pada `CounterProduksi`, buat tombol "Emergency Stop". Saat diklik, ubah state status menjadi "EMERGENCY", tombol "+1" menjadi disabled (`disabled`), dan tampilkan pesan merah.

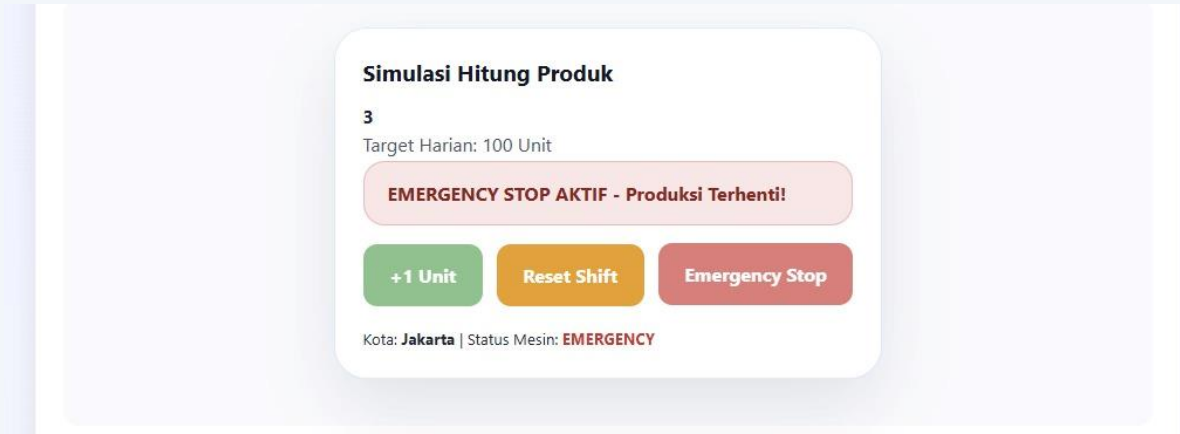
Jawaban / Kode Solusi:

```
import React, { useState } from 'react';
// Import React, useState (Pertemuan 10: State, Hooks)

function CounterProduksi({ kota = 'Jakarta' }) { // Komponen untuk menghitung
produksi dengan emergency stop (Latihan 2: Conditional Rendering)
  // State untuk jumlah produksi (Pertemuan 10: useState)
  const [jumlah, setJumlah] = useState(0); // Counter produksi awal 0

  // State untuk target harian (Pertemuan 10: useState)
  const [target, setTarget] = useState(100); // Target default 100 unit

  // State untuk nama kota (Pertemuan 10: useState) - menggunakan props dari
parent
  const [kotaState, setKotaState] = useState(kota); // State kota dengan
default value dari props

  // State untuk status mesin (Latihan 2: Emergency Stop)
  const [status, setStatus] = useState('NORMAL'); // Status: NORMAL atau
EMERGENCY

  // Fungsi menambah produksi (hanya jika status NORMAL) (Pertemuan 5:
Function)
  const tambahProduksi = () => {
    if (status === 'NORMAL') { // Pertemuan 4: Control Flow (if statement)
      setJumlah(jumlah + 1); // Tambah 1 unit
    }
  };

  // Fungsi reset counter (Pertemuan 5: Function)
  const reset = () => {
    setJumlah(0); // Reset ke 0
    setStatus('NORMAL'); // Reset status ke NORMAL
  };

  // Fungsi emergency stop (Latihan 2: Conditional Rendering)
  const emergencyStop = () => {
    setStatus('EMERGENCY'); // Ubah status ke EMERGENCY
  };

  return (
    <div className="card shadow-sm p-4 mb-3"> {/* Card Bootstrap dengan
padding */}
      <div className="card-body text-center">
        <h3 className="card-title">Simulasi Hitung Produk</h3>
```

```
        {/* Display angka produksi dengan ukuran besar */}
        <div className="display-3 fw-bold mb-2">{jumlah}</div> {/*
Menampilkan jumlah produksi */}

        <p className="text-muted">Target Harian: {target} Unit</p> {/*
Menampilkan target */}

        {/* Conditional Rendering: Pesan berdasarkan status (Pertemuan 10:
Conditional Rendering) */}
        {status === 'EMERGENCY' ? (
          // Jika status EMERGENCY, tampilkan pesan merah (Latihan 2)
          <div className="alert alert-danger fw-bold">
            EMERGENCY STOP AKTIF - Produksi Terhenti!
          </div>
        ) : jumlah >= target ? (
          // Jika jumlah >= target, tampilkan pesan sukses
          <div className="alert alert-success fw-bold">
            Target Tercapai! ({jumlah}/{target} Unit)
          </div>
        ) : (
          // Default: pesan produksi berjalan
          <div className="alert alert-secondary">
            Produksi Berjalan... ({jumlah}/{target} Unit)
          </div>
        )}

        {/* Buttons untuk kontrol (Pertemuan 6: Event Handling) */}
        <div className="mt-3 d-flex gap-2 justify-content-center flex-wrap">
          {/* Tombol +1 Unit - disabled jika emergency stop aktif (Latihan 2)
*/}
          <button
            className="btn btn-success btn-lg"
            onClick={tambahProduksi}
            disabled={status === 'EMERGENCY'} // Disabled saat EMERGENCY
(Latihan 2)
          >
            +1 Unit
          </button>

          {/* Tombol Reset */}
          <button className="btn btn-warning btn-lg" onClick={reset}>
            Reset Shift
          </button>

          {/* Tombol Emergency Stop (Latihan 2: Conditional Rendering) */}
          <button
```

```
            className="btn btn-danger btn-lg"
            onClick={emergencyStop}
            disabled={status === 'EMERGENCY'} // Disabled jika sudah dalam
status EMERGENCY
          >
            Emergency Stop
          </button>
        </div>

{/* Informasi status tambahan (Latihan 2) */}
        <div className="mt-3">
          <small className="text-muted">
            Kota: <strong>{kota}</strong> | Status Mesin: <strong
className={status === 'EMERGENCY' ? 'text-danger' : 'text-success'}>
              {status}
            </strong>
          </small>
        </div>
      </div>
    </div>
  );
}

export default CounterProduksi;
```

Screenshot Hasil:



Gambar F.2 – Hasil Latihan 2

F.4 Tugas Proyek Mini – (Kalkulator OEE Sederhana):

Deskripsi proyek mini:

a. Buat komponen form input: `Plan Time`, `Run Time`, `Total Parts`, `Good Parts`.
b. Gunakan State untuk menyimpan nilai-nilai ini.
c. Hitung OEE secara otomatis (Real-time) setiap kali input berubah.
d. Rumus OEE = (Availability x Performance x Quality).
e. Tampilkan hasil OEE dalam persen. Jika OEE < 50%, warnanya merah. Jika > 85%, warnanya hijau.

Penjelasan pendekatan/solusi yang digunakan:

Pada proyek mini ini, pendekatan yang digunakan berfokus pada penerapan konsep React.js, khususnya penggunaan state (useState), useEffect, serta conditional rendering dalam membuat kalkulator OEE (Overall Equipment Effectiveness) sederhana. Program ini dirancang untuk menerima input data produksi dari pengguna, kemudian menghitung nilai OEE secara otomatis dan menampilkannya secara langsung pada tampilan web.

Tahap awal dilakukan dengan merancang tampilan menggunakan React dan dibantu Bootstrap agar lebih rapi. Halaman terdiri dari judul kalkulator OEE dan beberapa input seperti Plan Time, Run Time, Total Parts, dan Good Parts. Input ini dibuat dalam bentuk form supaya mudah diisi oleh pengguna. Selain itu, ditampilkan juga hasil perhitungan OEE dalam bentuk persentase agar lebih mudah dipahami. Selanjutnya, logika program dibuat menggunakan state dengan useState. Setiap input disimpan ke dalam state yang berbeda. Jadi, setiap kali pengguna mengisi atau mengubah nilai, data tersebut langsung tersimpan dan diperbarui. Dengan cara ini, aplikasi bisa langsung merespon perubahan tanpa perlu reload halaman.

Perhitungan OEE dilakukan secara otomatis berdasarkan nilai input yang dimasukkan. Rumus yang digunakan adalah OEE = Availability × Performance × Quality. Karena data disimpan dalam state, setiap perubahan input akan langsung memicu perhitungan ulang dan hasilnya langsung tampil di layar. Ini membuat aplikasi terasa real-time. Untuk mendukung proses tersebut, digunakan useEffect agar sistem bisa berjalan otomatis saat ada perubahan data. Jadi, tanpa perlu tombol hitung, hasil OEE langsung muncul setiap kali input diubah. Hal ini membuat aplikasi lebih praktis dan efisien. Selain itu, digunakan conditional rendering untuk menampilkan hasil OEE dengan tampilan yang berbeda sesuai kondisi. Jika nilai OEE kurang dari 50%, maka ditampilkan dengan warna merah. Jika lebih dari 85%, ditampilkan dengan warna hijau. Sedangkan nilai di tengah ditampilkan dengan warna biasa. Cara ini membantu pengguna lebih cepat memahami apakah performa produksi sedang baik atau tidak.

Secara keseluruhan, proyek ini sudah menerapkan konsep utama pada pertemuan 10, yaitu penggunaan state untuk mengelola data, useEffect untuk proses otomatis, dan conditional rendering untuk mengatur tampilan berdasarkan kondisi. Hasilnya adalah aplikasi sederhana yang bisa menghitung OEE secara langsung dan menampilkan hasilnya dengan jelas, sehingga sudah mendekati sistem monitoring yang digunakan di dunia industri.

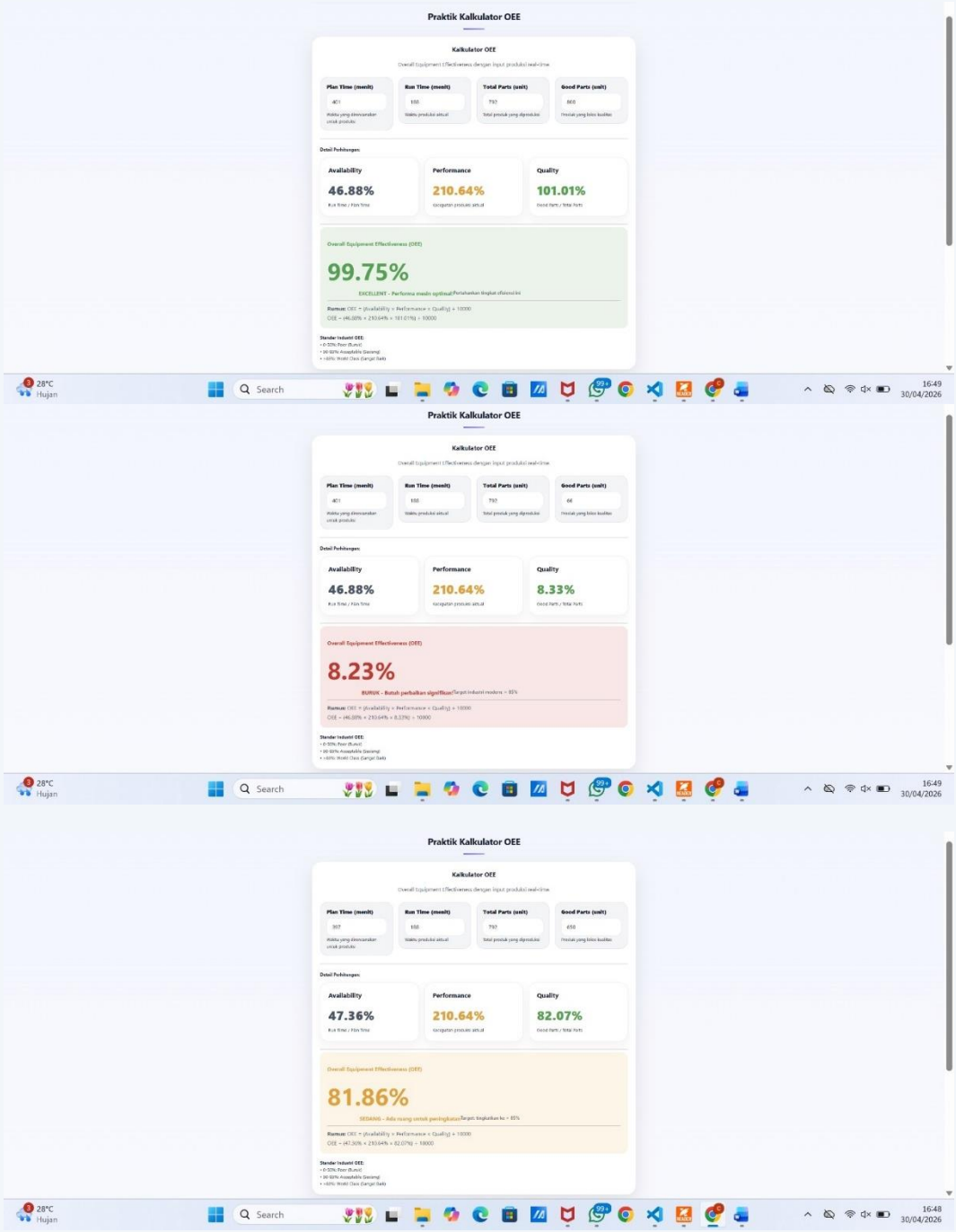
Kode Utama:

```
import React from 'react';
// Import React untuk JSX (Pertemuan 9) - Komponen utama
import KartuMesin from './kartumesin'; // Import komponen KartuMesin
(Pertemuan 9: React.js Component, JSX, Props)
import KartuKaryawan from './KartuKaryawan'; // Import komponen KartuKaryawan
import CounterProduksi from './CounterProduksi'; // Import komponen
CounterProduksi (Praktik E1)
import JamDigital from './JamDigital'; // Import komponen JamDigital (Praktik
E2)
import KalkulatorOEE from './KalkulatorOEE'; // Import komponen KalkulatorOEE

// Data mock untuk mesin produksi (Pertemuan 9: Props) - Array objek sebagai
data statis untuk demonstrasi
const mesinData = [
  { nama: 'CNC-Turning-01', status: 'Running', produksi: 150 },
  { nama: 'CNC-Milling-02', status: 'Maintenance', produksi: 0 },
  { nama: 'Press-Hydraulic-05', status: 'Stop', produksi: 85 },
];

// Data mock untuk karyawan (Pertemuan 9: Props) - Array objek untuk data
karyawan
const karyawanData = [
  { nama: 'Ari Prakoso', jabatan: 'Manager', bagian: 'Produksi' },
  { nama: 'Sinta Dewi', jabatan: 'Operator', bagian: 'Lini A' },
  { nama: 'Dedi Susanto', jabatan: 'QC', bagian: 'Kontrol Kualitas' },
];

function App() { // Komponen utama App (Pertemuan 9) - Fungsi komponen React
  const filteredMesin = mesinData; // Menampilkan semua mesin tanpa pencarian
  const filteredKaryawan = karyawanData; // Menampilkan semua karyawan tanpa
pencarian

  return (
    <div className="container"> {/* Container bootstrap-like untuk layout
responsif */}
      <header className="hero"> {/* Bagian hero untuk nama mahasiswa */}
        <div className="hero-card">
          <p className="student-name">Nailah Azka Iftinah</p>
          <p className="student-nim">NIM 23051430026</p>
        </div>
      </header>

{/* Section: Praktik Jam Digital */}
      <section className="section mb-4">
        <div className="card practice-card border-0 mb-4">
          <div className="card-body p-4 text-center">
```

```
<h3 className="fw-semibold mb-3">Praktik 1 useState untuk Counter
Produksi</h3>
<div className="practice-inner">
              <CounterProduksi kota="Jakarta" />
            </div>
          </div>
        </div>
      </section>

{/* Section: Praktik Counter Produksi */}
      <section className="section mb-4">
        <div className="card practice-card border-0 mb-4">
          <div className="card-body p-4">
            <h3 className="fw-semibold mb-3 text-center">Praktik 2 useEffect
untuk Real-time Clock</h3>
<div className="practice-inner">
              <JamDigital kota="Yogykarta" />
            </div>
          </div>
        </div>
      </section>

      {/* Section: Praktik Kalkulator OEE */}
      <section className="section mb-4">
        <h2 className="section-title">Praktik Kalkulator OEE</h2>
        <KalkulatorOEE />
      </section>

      {/* Section: Monitoring Mesin */}
      <section className="section"> {/* Semantic HTML: section untuk grup
komponen (Pertemuan 1) - Grouping untuk section mesin */}
        <h2 className="section-title">Monitoring Lini Produksi A</h2> {/*
Heading untuk section (Pertemuan 1) - Subjudul untuk section */}
        <div className="row"> {/* Row Bootstrap untuk grid layout (Pertemuan
3) - Grid system Bootstrap */}
          {filteredMesin.length > 0 ? ( // Conditional rendering jika ada
data (Pertemuan 10) - Render kondisional berdasarkan data
            filteredMesin.map((mesin) => ( // Mapping array untuk render
komponen (Pertemuan 5: Array, Object) - Iterasi array untuk render
              <div key={mesin.nama} className="col-md-4"> {/* Key untuk list
rendering, col untuk grid (Pertemuan 3) - Key unik untuk React, kolom grid
*/}
                <KartuMesin {...mesin} /> {/* Spread operator untuk props
(Pertemuan 9) - Passing props dengan spread */}
              </div>
            ))
```

```
          ) : (
            <p className="empty-state">Tidak ada mesin yang cocok.</p> //
Empty state (Pertemuan 10) - Pesan jika tidak ada data
          )}
        </div>
      </section>

      <section className="section"> {/* Semantic HTML: section untuk grup
komponen (Pertemuan 1) - Grouping untuk section karyawan */}
        <h2 className="section-title">Kartu Karyawan</h2> {/* Heading untuk
section (Pertemuan 1) - Subjudul untuk section */}
        <div className="row"> {/* Row Bootstrap untuk grid layout (Pertemuan
3) - Grid system Bootstrap */}
          {filteredKaryawan.length > 0 ? ( // Conditional rendering jika ada
data (Pertemuan 10) - Render kondisional
            filteredKaryawan.map((karyawan) => ( // Mapping array untuk
render komponen (Pertemuan 5) - Iterasi array
              <div key={karyawan.nama} className="col-md-4"> {/* Key untuk
list rendering, col untuk grid (Pertemuan 3) - Key unik dan kolom grid */}
                <KartuKaryawan {...karyawan} /> {/* Spread operator untuk
props (Pertemuan 9) - Passing props */}
              </div>
            ))
          ) : (
            <p className="empty-state">Tidak ada karyawan yang cocok.</p> //
Empty state (Pertemuan 10) - Pesan empty state
          )}
        </div>
      </section>
    </div>
  );
}

export default App;
```

Screenshot Hasil:



Gambar F.4 – Hasil Proyek Mini

G. PEMBAHASAN

Analisis hasil praktikum secara kritis. Jawab pertanyaan berikut dengan lengkap dan mendalam (bukan hanya "ya/tidak").

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum pertemuan 10 ini secara keseluruhan telah sesuai dengan teori yang dipelajari mengenai penggunaan state, hooks (useState dan useEffect), serta conditional rendering dalam pengembangan aplikasi React yang interaktif. Dari proses pengelolaan data hingga perubahan tampilan secara real-time, semua menunjukkan bahwa konsep dalam modul dapat diterapkan secara langsung dalam pengembangan aplikasi web modern.

Pada tahap awal praktikum, dilakukan pembuatan komponen CounterProduksi yang memanfaatkan useState untuk menyimpan data jumlah produksi dan target. Hal ini sesuai dengan teori bahwa state digunakan untuk menyimpan data yang dapat berubah selama aplikasi berjalan. Ketika tombol ditekan, nilai state akan diperbarui menggunakan fungsi setter, dan React secara otomatis melakukan re-render sehingga tampilan langsung berubah. Ini membuktikan bahwa konsep state sebagai “memori dinamis” dalam komponen telah diterapkan dengan baik. Selanjutnya, penggunaan fungsi seperti tambahProduksi dan reset menunjukkan bagaimana state dapat dimanipulasi berdasarkan interaksi pengguna. Hal ini sesuai dengan teori bahwa useState memungkinkan aplikasi menjadi interaktif karena data dapat berubah sesuai aksi user. Perubahan nilai jumlah produksi yang langsung terlihat pada tampilan menunjukkan bahwa mekanisme re-render React berjalan dengan benar.

Pada bagian conditional rendering, aplikasi menampilkan pesan yang berbeda berdasarkan kondisi jumlah produksi terhadap target. Jika jumlah produksi sudah mencapai target, maka ditampilkan pesan “Target Tercapai”, sedangkan jika belum maka ditampilkan “Produksi Berjalan”. Hal ini sesuai dengan teori bahwa conditional rendering digunakan untuk mengatur tampilan berdasarkan kondisi tertentu, sehingga aplikasi dapat memberikan informasi yang lebih relevan kepada pengguna. Pada bagian JamDigital, penggunaan useEffect menunjukkan penerapan konsep side effects dalam React. Fungsi setInterval digunakan untuk memperbarui waktu setiap detik, yang kemudian disimpan dalam state. Hal ini sesuai dengan teori bahwa useEffect dapat digunakan untuk menjalankan proses otomatis di latar belakang. Selain itu, penggunaan cleanup function dengan clearInterval juga telah sesuai dengan teori untuk mencegah terjadinya memory leak.

Pada bagian tambahan seperti form input pada KartuMesin, penggunaan state lokal untuk mengubah status mesin menunjukkan bahwa komponen dapat memiliki state sendiri yang berbeda dari props. Hal ini membuktikan bahwa React tidak hanya menggunakan data dari luar (props), tetapi juga mampu mengelola data internal secara fleksibel sesuai kebutuhan aplikasi.

Secara keseluruhan, praktikum ini menunjukkan bahwa konsep state, useState, useEffect, serta conditional rendering telah diterapkan dengan baik dalam satu aplikasi yang

interaktif. Hasil yang ditampilkan juga membuktikan bahwa aplikasi mampu merespon perubahan data secara langsung tanpa mengalami freeze atau reload halaman. Hal ini sesuai dengan tujuan pembelajaran pada pertemuan 10, yaitu membangun aplikasi React yang dinamis, responsif, dan mampu menangani perubahan data secara real-time.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Masih bingung alur penggunaan state dan hooks.	Mengatasi dengan latihan berulang dan mencoba berbagai variasi kode sampai memahami alurnya.
Bingung memahami perbedaan antara props dan state.	Mempelajari ulang materi dan mencoba langsung di kode agar lebih paham perbedaannya.

G.3 Kaitan dengan Penerapan di Industri

Materi praktikum pertemuan 10 ini sangat berkaitan dengan kebutuhan di bidang Teknik Industri, terutama dalam pembuatan sistem monitoring yang interaktif dan berbasis data real-time. Jika pada pertemuan sebelumnya fokus pada tampilan data, pada pertemuan ini lebih fokus pada bagaimana data bisa berubah dan langsung ditampilkan sesuai kondisi terbaru. Hal ini sesuai dengan kondisi di industri, di mana data seperti jumlah produksi atau status mesin selalu berubah dan harus terus diperbarui dalam sistem dashboard.

Penggunaan state dalam React menggambarkan kondisi nyata di industri yang selalu berubah. Misalnya, jumlah produksi akan terus bertambah selama proses berjalan. Dengan state, perubahan ini bisa langsung ditampilkan tanpa perlu reload halaman. Hal ini penting agar informasi yang ditampilkan selalu up-to-date dan bisa langsung digunakan untuk pengambilan keputusan. Konsep useState menunjukkan bagaimana sistem dapat merespon input atau perubahan data dengan cepat. Dalam dunia industri, perubahan bisa datang dari operator atau sensor mesin. Contohnya seperti menambah jumlah produksi atau melakukan reset data. Dengan useState, semua perubahan tersebut bisa langsung diproses dan ditampilkan secara otomatis.

Penggunaan useEffect juga sangat relevan karena menggambarkan sistem yang berjalan otomatis di latar belakang. Dalam industri, banyak sistem monitoring yang terus memperbarui data secara berkala, seperti sensor atau dashboard SCADA. Pada praktikum ini, hal tersebut terlihat dari jam digital yang terus berjalan setiap detik tanpa perlu dikontrol manual. Selain itu, conditional rendering mencerminkan sistem yang menampilkan informasi

berdasarkan kondisi tertentu. Misalnya, jika target produksi sudah tercapai, maka akan muncul pesan khusus. Jika belum, maka sistem menunjukkan bahwa produksi masih berjalan. Hal ini membantu pengguna memahami kondisi dengan cepat tanpa harus membaca data secara detail. Interaktivitas dalam aplikasi juga mencerminkan sistem industri modern yang mudah digunakan dan responsif. Pengguna bisa langsung berinteraksi dengan sistem, seperti menambah data atau melakukan reset. Hal ini membuat proses kerja menjadi lebih cepat dan efisien.

Walaupun pada praktikum ini masih berupa simulasi sederhana, konsep yang dipelajari sudah memberikan gambaran nyata tentang bagaimana sistem dashboard produksi dan monitoring real-time bekerja di industri. Bagi mahasiswa Teknik Industri, hal ini penting karena nantinya akan digunakan dalam sistem seperti dashboard produksi, sistem monitoring, dan aplikasi berbasis web untuk mendukung pengambilan keputusan.

H. KESIMPULAN

Tuliskan 3–5 poin kesimpulan yang merangkum hal-hal yang dipelajari dan dicapai pada praktikum modul ini. Kesimpulan harus spesifik, bukan bersifat umum.

1	Mahasiswa mampu memahami konsep state sebagai data yang dapat berubah dalam komponen React dan digunakan untuk menyimpan informasi sementara.
2	Mahasiswa mampu menggunakan useState untuk mengelola dan memperbarui data sehingga aplikasi menjadi interaktif.
3	Mahasiswa memahami penggunaan useEffect untuk menjalankan proses tambahan seperti timer atau update data secara otomatis.
4	Mahasiswa mampu menerapkan conditional rendering untuk menampilkan atau menyembunyikan tampilan berdasarkan kondisi tertentu.
5	Mahasiswa mampu mengembangkan aplikasi React yang dinamis, responsif, dan mampu menampilkan perubahan data secara real-time sehingga mendukung pembuatan sistem monitoring yang sesuai dengan kebutuhan industri.

I. DAFTAR PUSTAKA

Tuliskan semua sumber referensi yang digunakan dalam format APA 7th Edition. Minimal 3 referensi yang relevan (termasuk Manual Praktikum dan sumber online resmi seperti MDN, Reactjs.org, dll).

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Hakim, R., Carudin, C., & Umaidah, Y. (2025). Implementasi React JS dan Integrasi Strapi CMS dalam Pengembangan Website OK OCE Indonesia. Jurnal Mahasiswa Teknik Informatika (JATI).
[3]	Parabi, M. I., Ilman, I. S., Rahmawati, A. N., & Putra, C. P. (2025). Pengembangan Antarmuka Sistem Kinerja Dosen Menggunakan React JS. Jurnal Pendidikan Vokasi Teknik Informatika.
[4]	Akbar, R. M., Zahara, S., Sukmaningtyas, Y. N., Syaifuddin, A., & Putra, H. F. (2026). Implementasi React JS pada Marketplace Desa Kalikatir Mojokerto. Jurnal Informatika dan Teknik Elektro Terapan.
[5]	Rahmadhani, S., Wildana, D. W., Arumdanie, H. W., & Hakim, L. (2024). Penerapan React JS dan Axios untuk Pengembangan Front-end Aplikasi iCare. Jurnal Pengembangan Teknologi Informasi dan Ilmu Komputer.

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	**Aspek Penilaian**	**Bobot (%)**	**Nilai (0-100)**	**Skor Akhir**	**Catatan Penilai**
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
	TOTAL NILAI AKHIR	**100%**			

Mahasiswa	Diperiksa oleh Asisten / Dosen
 _____ Nailah Azka Iftinah NIM:23051430026	_____ Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____